TypeScript Fundamentals

The topics are:

1. Type inference
2. Interfaces vs. Types
3. Generics
4. Utility types — Partial, Pick, Record
5. Discriminated unions
6. satisfies operator

1. Type Inference

Type inference means TypeScript automatically determines the type of a variable from its value.

Example

```
let name = "Rahul";
let age = 25;
let isStudent = true;
```

TypeScript automatically understands:

name → string

age → number

isStudent → boolean

Therefore:

name = 100;

produces an error:

Type 'number' is not assignable to type 'string'.

Function inference

```
function add(a: number, b: number) {
  return a + b;
}
```

TypeScript infers the return type: **number**

So this is equivalent to:

function add(a: number, b: number): number

{ return a + b; }

When inference is useful

You don't need to write types everywhere.

Instead of:

const city: string = "Pune";

const temperature: number = 28;

you can usually write:

const city = "Pune";

const temperature = 28;

2. Interfaces vs Types

Both interface and type can describe the structure of objects.

Interface

```
interface Student {
  id: number;
  name: string;
  email: string;
}
```

Use it:

```
const student: Student = {
  id: 101,
  name: "DES",
  email: "DESPU@despu.edu.in"
};
```

Type

The same structure can be represented using a type:

```
type Student = {
  id: number;
  name: string;
  email: string;
};
```

For normal object structures, both work very similarly.

Important difference

Interfaces can be **extended**:

```
interface Person {
  name: string;
}

interface Student extends Person {
  rollNo: number;
}
```

Now:

```
const student: Student = {
  name: "Rahul",
  rollNo: 101
};
```

Types can also be composed using intersections:

```
type Person = {
  name: string;
};

type Student = Person & {
  rollNo: number;
};
```

Types are more flexible

A type can represent things that aren't just object structures.

For example:

```
type ID = string | number;
```

or: type Status = | "pending" | "approved" | "rejected";
or:
type Point = [number, number];

3. Generics

Generics allow us to write **reusable, type-safe code**.

Consider:

```
function identity(value: number): number {
  return value;
}
```

This only works with numbers.

We could create another function:

```
function identityString(value: string): string {
  return value;
}
```

That's duplication.

Instead, use a generic:

```
function identity<T>(value: T): T {
  return value;
}
```

Now:

```
const a = identity<number>(100);
const b = identity<string>("Hello");
const c = identity<boolean>(true);
```

TypeScript understands:

a → number

b → string

c → boolean

Usually TypeScript can infer T:

```
const a = identity(100);
const b = identity("Hello");
```

Generic interface

```
interface ApiResponse<T> {
  data: T;
  status: number;
}
```

Use it:

```
interface Weather {
  city: string;
  temperature: number;
```

```
}

const response: ApiResponse<Weather> = {
  data: {
    city: "Pune",
    temperature: 28
  },
  status: 200
};
```

This is extremely useful for API applications.

4. Utility Types

TypeScript provides built-in utility types for transforming existing types.

The three important ones here are:

Partial

Pick

Record

4.1 Partial

Suppose:

```
interface Student {
  id: number;
  name: string;
  email: string;
  age: number;
}
```

Normally all properties are required:

```
const student: Student = {
  id: 1,
  name: "Amit",
  email: "amit@example.com",
  age: 24
};
```

But suppose you're updating only the email.

You can use:

```
const update: Partial<Student> = {
  email: "newemail@example.com"
};
```

Partial<Student> makes **all properties optional**.

Conceptually:

```
{
```

```
  id?: number;
  name?: string;
  email?: string;
  age?: number;
}
```

Common use

Partial is especially useful for:

Update operations

Form data

Configuration overrides

PATCH APIs

4.2 Pick

Suppose:

```
interface Student {
  id: number;
  name: string;
  email: string;
  age: number;
  address: string;
}
```

But you only need:

id

name

email

Use:

```
type StudentSummary =
  Pick<Student, "id" | "name" | "email">;
```

Now:

```
const student: StudentSummary = {
  id: 101,
  name: "Rahul",
  email: "rahul@example.com"
};
```

Pick means:

Select specific properties from an existing type.

4.3 Record

Record is useful when you need an object with a known set of keys.

Example:

```
type Role = "admin" | "teacher" | "student";
```

Now:

```
const permissions: Record<Role, boolean> = {
  admin: true,
  teacher: true,
  student: false
};
```

TypeScript requires all three keys.

If you forget:

```
student: false
```

TypeScript reports an error.

Another example:

```
type WeatherStatus =
  "sunny" |
  "cloudy" |
  "rainy";

const icons: Record<WeatherStatus, string> = {
  sunny: "☀️",
  cloudy: "☁️",
  rainy: "🌧️"
};
```

This is particularly useful in a **weather dashboard**.

5. Discriminated Unions

This is one of the most powerful TypeScript features for application state.

Suppose an API can have four states:

idle

loading

success

error

Create:

```
type ApiState<T> =
  | {
      status: "idle";
    }
  | {
      status: "loading";
    }
  | {
      status: "success";
```

```
    data: T;
   }
  | {
    status: "error";
    error: string;
   };
```

Here:

status

is the **discriminant**.

Using it

```
type WeatherState =
  ApiState<Weather>;
```

Then:

```
function displayWeather(
  state: WeatherState
) {
  switch (state.status) {

    case "idle":
      console.log("Search for a city");
      break;

    case "loading":
      console.log("Loading...");
      break;

    case "success":
      console.log(
        state.data.temperature
      );
      break;

    case "error":
      console.log(state.error);
      break;
  }
}
```

Notice something important.

Inside:

```
case "success":
```

TypeScript knows that:
state.data
exists.
Inside:
case "error":
TypeScript knows:
state.error
exists.
This is called **type narrowing**.

6. satisfies Operator
The satisfies operator was introduced in **TypeScript 4.9**.
Suppose we have:

```
type WeatherConfig = {
  apiUrl: string;
  units: "metric" | "imperial";
  refreshInterval: number;
};
```

We can write:

```
const config = {
  apiUrl: "/api/weather",
  units: "metric",
  refreshInterval: 60000
} satisfies WeatherConfig;
```

TypeScript verifies that config satisfies WeatherConfig.

Why is this useful?
Consider:

```
const config: WeatherConfig = {
  apiUrl: "/api/weather",
  units: "metric",
  refreshInterval: 60000
};
```

This works, but the variable's type is explicitly widened to WeatherConfig.
With:

```
const config = {
  apiUrl: "/api/weather",
  units: "metric",
  refreshInterval: 60000
} satisfies WeatherConfig;
```

TypeScript **checks the shape against WeatherConfig while preserving the inferred type of the expression**.

Error detection

This:

```
const config = {
  apiUrl: "/api/weather",
  units: "kelvin",
  refreshInterval: 60000
} satisfies WeatherConfig;
```

produces an error because:

"kelvin"

isn't allowed.

8. A small practical exercise for All TY C DES

Give students this JavaScript:

```
const students = [
  {
    id: 1,
    name: "Wasudeo",
    department: "CSE"
  },
  {
    id: 2,
    name: "Rahane",
    department: "ECE"
  }
];

function getStudent(id) {
  return students.find(s => s.id === id);
}
```

Ask them to refactor it into strict TypeScript and demonstrate:

Task 1 — Interface

```
interface Student {
  id: number;
  name: string;
  department: string;
}
```

Task 2 — Generic function

```
function findById<T extends { id: number }>(
  items: T[],
```

```
  id: number
): T | undefined {
  return items.find(item => item.id === id);
}
```

Task 3 — Pick

```
type StudentSummary =
  Pick<Student, "id" | "name">;
```

Task 4 — Partial

```
type StudentUpdate =
  Partial<Student>;
```

Task 5 — Record

```
const departments: Record<
  string,
  number
> = {
  CSE: 45,
  ECE: 38
};
```

Task 6 — Discriminated union

```
type StudentStatus =
  | { status: "active" }
  | { status: "inactive"; reason: string };
```

Task 7 — satisfies

```
const settings = {
  maxStudents: 60,
  department: "CSE"
} satisfies {
  maxStudents: number;
  department: string;
};
```